AI ARCHITECTURE DESIGN · SESSION 1 · STUDY GUIDE

Architecture Views, Steps and Process

A detailed companion to the session slides. It explains each idea and then applies it, step by step, to the course's running project: **Marketing Automation with AI & Agents**.

Course	AI Architecture Design (Unifonic · MLOps MENA)
Instructor	Eng. Sameh Amin
Session date	18 October 2026
Source	Session slide deck (36 slides). The recording itself could not be accessed, so this guide follows the slides. The expanded explanations and worked examples are additions, not quotes from the talk.

CONTENTS

1. The course at a glance
2. The architect's mindset
3. The running project & its business requirements
4. Architecture frameworks: TOGAF, Zachman, C4
5. The architecture development process
6. Viewpoints and perspectives
7. Architecture in an Agile team
8. Writing the architecture description (worked end to end)
9. Architecture governance
10. Architecture and agentic coding
11. Week 1 mini project: a worked starter
12. Glossary & self-check questions

CHAPTER 1

The course at a glance

Slides 1–5, 30–35

The course teaches a **general architecture method** and then applies it to AI systems. Session 1 covers the method itself: how to think, which frameworks to use, and what an architecture document contains. Sessions 2–6 then apply that method to AI-specific building blocks. *Analogy:* an ML engineer builds an excellent engine, and the architect designs the whole car around it: fuel line (data pipelines), dashboard (observability), brakes (guardrails), crash rating (security and compliance) and servicing (MLOps).

#	Session	What it covers
1	**Architecture Views, Steps and Process**	The architect's mindset, frameworks (TOGAF, Zachman, C4), the development process, viewpoints and perspectives, the architecture description, governance. *This guide.*
2	AI System Building Blocks	Data pipelines and feature stores; model layer (foundation, fine-tuned, classical ML); training vs inference infrastructure; serving, APIs, orchestration; MLOps/LLMOps and observability.
3	LLM & RAG Architecture	LLM selection and hosting (API vs self-hosted); prompting and context management; embeddings and vector DBs; the RAG pipeline (chunking, retrieval, re-ranking); hybrid search and GraphRAG.
4	Agentic AI Architecture	Agent patterns (ReAct, planner-executor); tools, function calling and MCP; memory and state; multi-agent orchestration; human-in-the-loop; guardrails.
5	Production Readiness	AI threat modeling (prompt injection, data leakage); guardrails and access control; governance and responsible AI; evals (offline, LLM-as-judge, human); production monitoring.
6	Capstone / Case Studies	Applied AI in telecom, e-commerce and government; lessons learned and trade-offs; capstone presentation.

What the course is

- A general architecture method, applied to AI systems.
- AI design best practices.
- Practical enterprise case studies (telecom, e-commerce, government) in every session.
- Explicit attention to **functional and non-functional quality attributes**, so designs are production-ready.
- Audience: senior engineers of any specialization (BE, FE, AI), EMs, TLs up to CTOs, plus semi-technical people and PMs.

What the course is not

- Not a product or business-ideas course, and not aimed at business-only audiences.
- Not a machine learning or data science course. You won't train models here; you'll decide *where* models sit, how data reaches them, and how they are run, secured and governed.

Toolkit

- Diagrams: draw.io, Mermaid, or paper.
- Wiki: Confluence, Obsidian, MS Word, etc.
- Excel or LibreOffice (capacity and cost sheets).
- An agentic coding IDE (optional).

CHAPTER 2

The architect's mindset Slide 6

The slide lists eight traits. Each one describes a behaviour you can observe in practice. Below, each trait is paired with what it looks like on the Marketing Automation project.

Trait	Meaning	On the Marketing Automation project
Humble	Puts the best solution ahead of their own idea.	You proposed a custom recommender. A data engineer shows that the CDP vendor's built-in "next-best-offer" module covers 70% of the need. You adopt it for the MVP.
Curious	Keeps learning new technologies *and* domains.	You learn how campaign managers actually work (segments, frequency caps, A/B tests) before designing the "campaign agent".
Pragmatic	Picks the simplest design that meets the requirements (MVP).	Cold start v1 = "top sellers in the visitor's region". No contextual bandit until data proves it's needed.
Technology-agnostic	Not attached to a favourite stack.	The vector DB choice is made by scoring options against requirements (an ADR), not by "we always use X".
Trade-off thinker	No design is perfect, and every choice has a cost.	Real-time personalization (<100 ms) means paying for an online feature store, so nightly batch scores are kept for email campaigns where latency doesn't matter.
Strong communicator (viewpoints)	Explains the same design to engineers and executives.	The CMO sees a one-box C1 context diagram. The platform team sees the C2 deployment view with ports and pods.
Collaborative	Listens first and influences without authority.	You don't own the CRM team's roadmap, so you persuade them to expose a consent API by showing how it unblocks *their* PDPL audit.
Accountable	Owns decisions and their consequences (POC, skeleton).	You build a walking skeleton (event → feature store → ranker → homepage) to prove the latency budget before the team commits.

KEY IDEA: "POC, SKELETON"

Accountability is shown in running code, not only in documents. A **proof of concept** answers "can this work at all?". A **skeleton system** (also called a walking skeleton) is a thin, end-to-end implementation of the architecture that touches every major component. It is production-shaped, but has almost no features. It shows that the architecture holds together before features are piled on. Chapter 5 returns to it as the final step of the process.

CHAPTER 3

The running project & its business requirements

Slides 7–9

Every session uses the same project: an e-commerce company wants **Marketing Automation with AI & Agents**. The slide splits the requirements into three groups, each answering a different question.

Functional	KPIs / Objectives	Non-functional
WHAT does it do?	HOW do we know it works?	Within WHAT limits?
use cases, features	measurable targets	quality attributes, constraints

3.1 Functional use cases

Below, each use case is traced as input → processing → output, using a store that sells seeds, fertilizers and farm tools.

(a) Homepage / product page personalization (feature stores)

```
INPUT    GET /homepage   user_id = 482
STEP 1   feature_store.get_online("user:482")
         → { last_purchase: "tomato seeds", region: "Minya",
             avg_basket_egp: 650, visits_7d: 4 }
STEP 2   ranker.score(user_features, candidate_products[500])
STEP 3   business_rules.apply(in_stock = true, margin ≥ 15%)
OUTPUT   [ "Tomato NPK fertilizer", "Drip irrigation kit", "Pruning shears", … ]
BUDGET   whole call < 100 ms  → features must be PRE-computed
```

A **feature store** keeps precomputed features about users and products in two forms. The *online* store is a fast key-value lookup used at request time. The *offline* store is historical and used for training. Because both are fed by the same definitions, the model sees the same "visits_7d" in training as in production. This prevents a common bug known as training/serving skew.

(b) Next-best action (NBA)

```
INPUT    { user: 482, event: "cart_abandoned", cart_value: 900,
           discount_sensitivity: "high", messages_this_week: 1 }
CANDIDATES  [ coupon_10pct, reminder_no_discount, bundle_offer, do_nothing ]
MODEL    P(convert | action) → [0.31, 0.29, 0.18, 0.12]
UPLIFT   coupon costs 90 EGP for +2 pts of conversion → not worth it
OUTPUT   reminder_no_discount  via WhatsApp  at 19:30
```

NBA picks the single most valuable action for this user right now. The value includes cost and fatigue, not only the probability of a click.

(c) New users (cold start)

```
INPUT    anonymous visitor, no user_id, no history
SIGNALS  device = mobile, geo = Assiut, referrer = "fb_ad_wheat_season",
         local_time = 06:40
OUTPUT   "Top wheat-season products in Upper Egypt"
         + one onboarding question: "What do you grow?"
```

(d) Agentic marketing: campaign automation

```
GOAL     "Clear excess potassium fertilizer stock before winter"
AGENT    1. query warehouse  → users who bought potato seeds 60–120 days ago (12,400)
         2. draft 3 message variants (Arabic / English; SMS / WhatsApp)
         3. ▮ HUMAN APPROVAL of copy + audience       ← business control
         4. launch A/B test on 10% of audience
         5. monitor 48 h, shift traffic to winning variant
OUTPUT   campaign report + updated feature: "responded_to_potassium_campaign"
```

3.2 KPIs / Objectives: "L2 connected to goals"

KPIs are layered. **L1** is a company goal (for example, "grow online revenue"). **L2** KPIs are measurable levers that roll up to that goal, and the system is designed to move them.

KPI	Worked meaning	Architecture consequence
Increase sales take-rate by 80%	If 5 of every 100 recommendations are bought today (5%), a *relative* 80% increase means 9%. Confirm with the business whether they mean relative (5% → 9%) or absolute (reach 80%), because these are very different targets.	You need event tracking that links each purchase back to the recommendation that caused it (attribution).
Increase CSAT to 70%	70% of survey answers are 4–5 out of 5. This guards against "sell more by spamming".	Frequency caps and opt-out handling become hard requirements.
Feedback loop	Every outcome (clicked, ignored, bought, unsubscribed) flows back into features and model retraining.	Event stream → feature store → scheduled retraining → model registry.

3.3 Non-functional requirements

(The slide labels these "non-functional use cases". The standard term is **non-functional requirements** or **quality attributes**.)

Requirement	Concrete meaning for this system
Latency: real-time / near real-time / batch	**Real-time** (<100 ms): homepage ranking. **Near real-time** (seconds to minutes): react to cart abandonment. **Batch** (nightly): audience building, retraining.
Scalability	Normal: ~5k sessions/hour. Peak campaign (planting season, Ramadan, White Friday): ~50k/hour. The design must survive 10× load.
Explainability & business control	Marketing can ask "why did user 482 get this offer?" and set guardrails such as "max 2 messages/week" or "never discount below 15% margin".
Security, compliance & privacy (PDPL)	Consent before using personal data for targeting, data minimization, retention limits and opt-out. Egypt's Personal Data Protection Law is Law 151/2020, and Saudi Arabia also has a PDPL. Which one applies depends on where customers are.

3.4 Turning requirements into a plan Slide 9

The slide poses five questions that every architect faces at the start. Here they are with practical answers:

1. **Who are the stakeholders and personas?** For example: CMO (sponsor), campaign managers (daily users), data engineering (owns pipelines), CRM team (owns customer data), legal/DPO (privacy), platform/SRE (runs it), and the end customer. Each will care about a different view (Chapter 6).
2. **Use-case assessment, "the planning call".** Score each use case on business value, feasibility, data readiness and risk, then decide what goes into the MVP. *Example:* homepage personalization = high value and data ready → MVP. Agentic campaigns = high value but high risk → phase 2, behind human approval.
3. **Should you give timelines now?** Only for items that are **ready for implementation**. For unknowns, commit to a **spike**: a time-boxed investigation, for example "3 days: can the CDP stream events with <2 s lag?". You give a date for the spike's answer, not for the feature.
4. **Should architecture design stop until all spikes finish?** Generally no. Design around the unknowns with explicit, written **assumptions**, mark the affected parts as provisional, and revisit them when the spike reports back. Stopping everything turns architecture into a bottleneck.
5. **Close the gaps in 1–n rounds.** Keep a running list of questions and answers with stakeholders. Expect several rounds, because each answer exposes new questions.

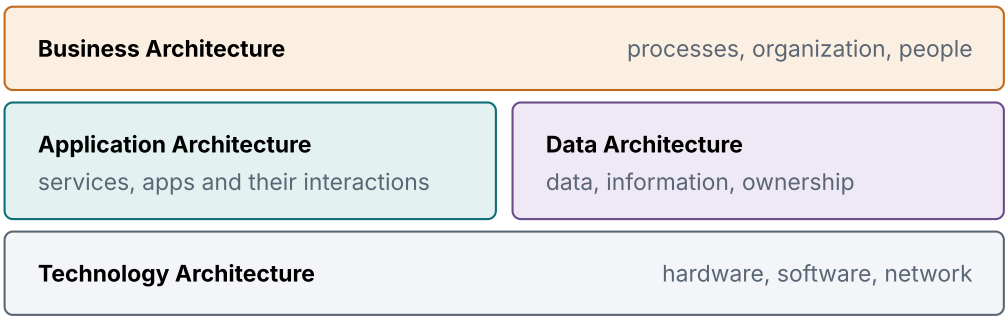
CHAPTER 4

Architecture frameworks Slides 10–14

A framework is a ready-made structure for thinking, so you don't forget a dimension. The session covers three, which operate at different altitudes: TOGAF (enterprise *process*), Zachman (enterprise *classification*), and C4 (software *diagrams*).

4.1 TOGAF: The Open Group Architecture Framework

TOGAF is a **method** for creating and delivering enterprise architectures. It splits architecture into four stacked domains, each resting on the one below:



The four TOGAF architecture domains (redrawn).

Its engine is the **ADM (Architecture Development Method)**, an iterative cycle. In outline: *Preliminary* (set up the practice) → *A. Vision* → *B. Business* → *C. Information Systems (data + application)* → *D. Technology* → *E. Opportunities & Solutions* → *F. Migration Planning* → *G. Implementation Governance* → *H. Change Management*, with *Requirements Management* at the centre feeding every phase.

> **APPLIED TO OUR PROJECT**
>
> - **Business:** campaign managers' workflow, the approval process for AI-generated copy, and KPIs.
> - **Data:** customer profile (CRM), behavioural events, consent records, product catalogue, and who owns each.
> - **Application:** recommender service, NBA service, campaign agent, CMS/homepage, messaging gateway.
> - **Technology:** Kubernetes cluster, GPU nodes for model serving, Kafka, Redis, cloud region choice for data residency.

4.2 Zachman Framework

Zachman is not a process. It is a **classification grid** (an ontology) that answers "what artifacts should exist?". It crosses six questions with six audiences. Each cell is a distinct model, and a complete enterprise description fills every cell.

	What data	How function	Where network	Who people	When time	Why motivation

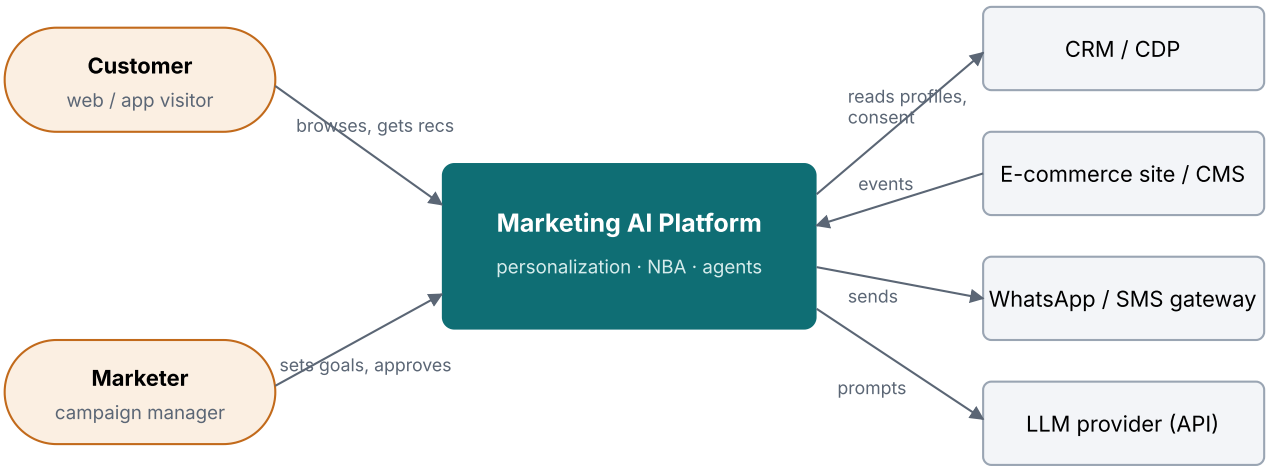
Planner scope	Key business things	Key processes	Business locations	Key organizations	Key events	Goals & strategy
Owner business model	Semantic model	Business process model	Logistics network	Workflow model	Master schedule	Business plan
Designer system model	Logical data model	Application architecture	Distributed system arch.	Human interface arch.	Processing structure	Business rule model
Builder technology	Physical data model	System design	Technology architecture	Presentation arch.	Control structure	Rule design
Implementer detailed	Data definition	Program	Network architecture	Security architecture	Timing definition	Rule specification
User functioning	Actual data	Working function	Actual network	Actual organization	Actual schedule	Actual strategy

> **HOW TO USE IT IN PRACTICE**
>
> Use the grid as a **gap checklist**. For our project, take the "Why" column. Planner row: "grow online revenue". Owner: "take-rate +80%, CSAT 70%". Designer: business rules such as "max 2 messages/week". Builder: the rule engine config. Implementer: the actual rule code. If any cell is empty, someone will eventually make that decision implicitly, and probably badly.

4.3 The C4 model

Created by Simon Brown and described as "maps of your code". C4 gives **four zoom levels**, from big picture to code, with one level per audience. It is **notation-free**: boxes, lines and labels, with no UML exam required. Supplementary diagrams include the system landscape, dynamic (runtime) and deployment views. Tools: Structurizr, Mermaid, draw.io.

Level	Shows	Audience	Our project
1. System Context	People, *the* system as one box, external systems	Everyone, incl. execs	Customer, Marketer ↔ Marketing AI Platform ↔ CRM, CDP, WhatsApp/SMS gateway, LLM provider
2. Containers	Deployable/runnable units: apps, APIs, DBs, queues	Architects, devs, ops	Recommendation API, NBA service, Campaign Agent, Feature Store, Kafka, Vector DB, Postgres
3. Components	Modules inside one container	Devs on that container	Inside Campaign Agent: Planner, Audience Tool, Copywriter, Approval Gate, Launcher
4. Code (optional)	Classes, ERD, sequence	Devs	Usually generated from the code, not hand-drawn

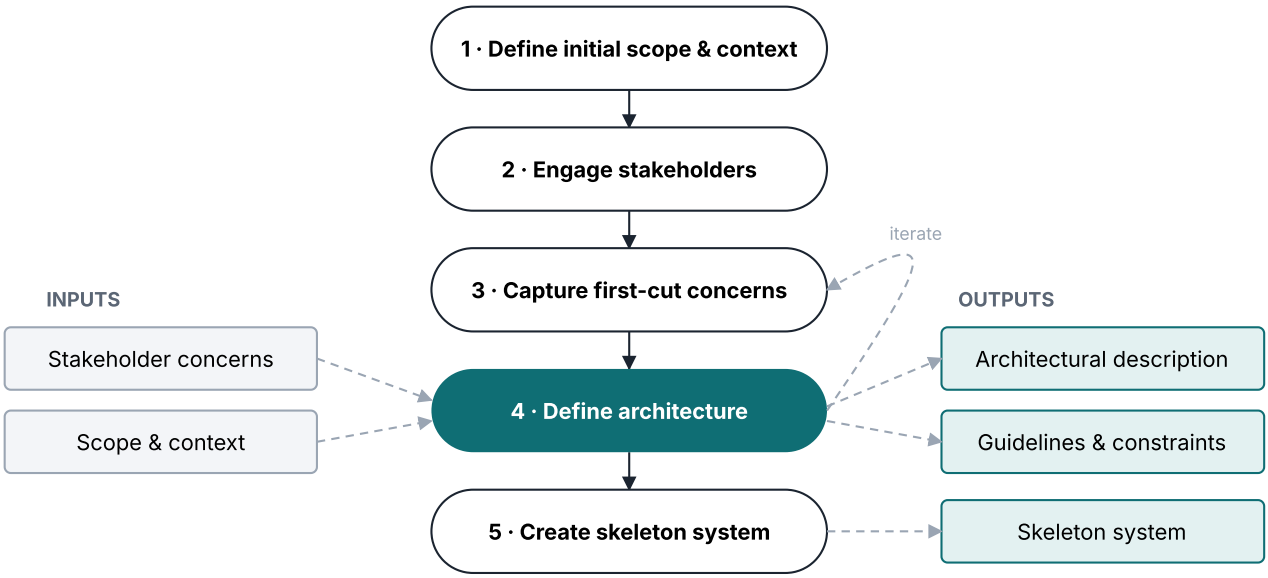


C4 Level 1: System Context for the Marketing Automation platform. One box for "our" system, with everything else outside the boundary.

CHAPTER 5

The architecture development process Slide 15

The slide shows a linear flow with inputs on the left and outputs on the right. It is iterative in practice: new concerns loop you back to "Define Architecture".



The architecture development process (redrawn and simplified). Step 4 is where the views and perspectives of Chapter 6 are applied.

Step	What you actually do (Marketing Automation)	Output
1. Define initial scope & context	Draw the first C1 context diagram. Write down in scope (web/app personalization, WhatsApp + SMS campaigns) and out of scope (call-centre scripts, offline stores).	Scope statement, C1 draft
2. Engage stakeholders	Interview the CMO, two campaign managers, CRM owner, DPO and SRE lead. Produce a stakeholder map (influence × interest).	Stakeholder list & map
3. Capture first-cut concerns	CMO: "prove uplift". DPO: "consent before targeting". SRE: "no new on-call nightmares". Marketers: "I must be able to stop a campaign instantly".	Concerns list → becomes requirements and perspectives
4. Define architecture	Produce the views (functional, information, concurrency, deployment, operational) and check each against perspectives (security, performance, availability, evolution).	Architecture description, ADRs, guidelines
5. Create skeleton system	Event from website → Kafka → feature store → toy ranker → homepage widget, deployed on the real cluster with real auth and real dashboards.	Running skeleton that proves the latency budget and wiring

WHY A SKELETON, NOT MORE DOCUMENTS?

A document can claim "<100 ms end-to-end". A skeleton *measures* it. If the measured p95 is 340 ms because the feature store sits in another region, you learn it in week 3 instead of week 20. This is the "Accountable" trait from Chapter 2 in practice.

CHAPTER 6

Viewpoints and perspectives

Slides 16–17

These two ideas solve two different problems. **Views** split the system into separate drawings so each audience can understand its part. **Perspectives** are quality concerns that cut across every view.

> **ANALOGY: A BUILDING**
>
> An architect produces a floor plan, an electrical plan, a plumbing plan and an elevation. Those are **views**: same building, different slices. Fire safety is not its own drawing. It is checked *on every* plan: exits on the floor plan, breakers on the electrical plan, sprinklers on the plumbing plan. Fire safety is a **perspective**.

6.1 The views

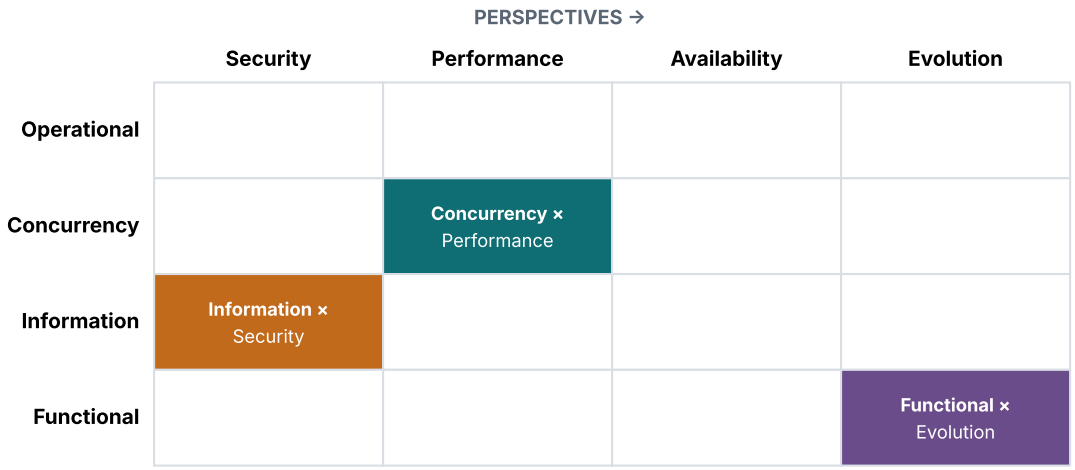
View	Question it answers	Our project
Functional	What are the runtime elements, their responsibilities and interfaces?	Recommendation API, NBA service, Campaign Agent, Approval UI, and their APIs.
Information	How is data stored, moved, owned, and kept correct?	Events → Kafka → feature store; consent in CRM; retention 13 months.
Concurrency	What runs in parallel, and how is it coordinated?	Sync ranking calls vs async campaign jobs; LLM rate limits; retries/DLQ.
Development	How is the code organized, built and tested?	Monorepo layout, shared feature definitions, CI checks, prompt versioning.
Deployment	What runs where, on which hardware and network?	K8s namespaces, GPU node pool for vLLM, Redis cluster, VPC rules.
Operational	How is it run, monitored and supported in production?	Grafana/Loki/Prometheus, Langfuse traces, on-call runbooks, model rollback.

6.2 The perspectives (quality attributes)

Typical perspectives are **security, performance & scalability, availability & resilience, evolution, usability, accessibility, location** (where users and data are) and **regulation**. You choose the ones that matter for *this* system. For Marketing Automation the critical four are security/privacy, performance, availability and regulation (PDPL).

6.3 Applying perspectives to views: the matrix

Cross every perspective with every view. Most cells hold a short note. A few cells are where the real design risk sits, and those deserve deep work. The slide highlights three classic "hot cells":



Perspectives × views. Shaded cells are the three examples highlighted on the slide (redrawn).

Hot cell	Generic concern	On our project
Concurrency × Performance	Shared resources, blocking, queuing, coordination.	At 50k sessions/hour, if every homepage call waits on a synchronous LLM call, the LLM quota becomes the bottleneck. Fix: no LLMs in the real-time path. Pre-generate copy asynchronously and cache it.
Information × Security	Access control, access classes, object-level security.	The Campaign Agent can query the warehouse. It must see *segments and counts*, never raw phone numbers. Phone numbers are resolved only inside the messaging gateway. PII never enters prompts.
Functional × Evolution	Extension points, flexible interfaces, meta-approaches.	Define NBA "actions" as a plug-in registry (`coupon`, `reminder`, `bundle`...) so marketing can add "loyalty points" next quarter without redesigning the service.

CHAPTER 7

Architecture in an Agile team

Slide 18

Architecture is not a single up-front phase. It happens at four levels of granularity, each with a different artifact and time horizon.

Level	Architect's job	Artifact	Example
Project	Overall shape, principles, major technology choices.	Architecture description (Chapter 8), C1/C2, principles	"Event-driven platform; LLMs only in async paths; consent checked centrally."
Epic	Design one capability across containers.	Epic design note, C3 of affected containers, ADRs	Epic "Cold start": components, data needed, fallback order.
Story	Make sure the story respects the architecture; answer team questions.	Acceptance criteria incl. NFRs	"Endpoint returns in <80 ms p95 and logs a recommendation_id for attribution."
Research / spike	Time-boxed investigation of an unknown.	Spike report → usually an ADR	"3 days: compare Qdrant vs pgvector on 2M product embeddings, filter by stock."

RULE OF THUMB

Decide early only what is **expensive to change later**, such as data ownership, sync vs async boundaries, and cloud region for data residency. Defer the rest to epic or story level, where you'll know more.

CHAPTER 8

Writing the architecture description Slides 19–24

This chapter is the practical core of the session: what goes into the architecture document, section by section. Each section below is filled in for the Marketing Automation project as a worked example.

8.1 Introduction and facts Slide 20

Section	Worked example
1. Problem statement / pain points	Every visitor sees the same homepage. Campaigns are built manually in spreadsheets and take 5 days. Discounts are sent to customers who would have bought anyway.
2. Goals / non-goals, assumptions	**Goals:** personalized homepage, NBA for cart abandonment, agent-assisted campaigns. **Non-goals:** pricing optimization, call-centre integration. **Assumptions:** the CDP can stream events with <2 s lag (spike pending); customers have consented to marketing for ~60% of the base.
3. As-is architecture	Monolithic e-commerce site, a CRM, a CDP collecting clickstream, a nightly ETL to the warehouse, and a third-party SMS tool. Also explore the *enterprise* architecture: what shared platforms (IAM, Kafka, observability) already exist that you must reuse?
4. Principles, key decisions, trade-offs	"Consent is checked once, centrally." "No LLM in the synchronous user path." "Buy before build for commodity capabilities." Each key decision links to an ADR.
5. Assessment	Gap analysis of as-is vs to-be: no online feature store, no event streaming from the site, no model registry, no consent API.
6. Priorities	P1 events + feature store + homepage recs → P2 NBA → P3 campaign agent with human approval.

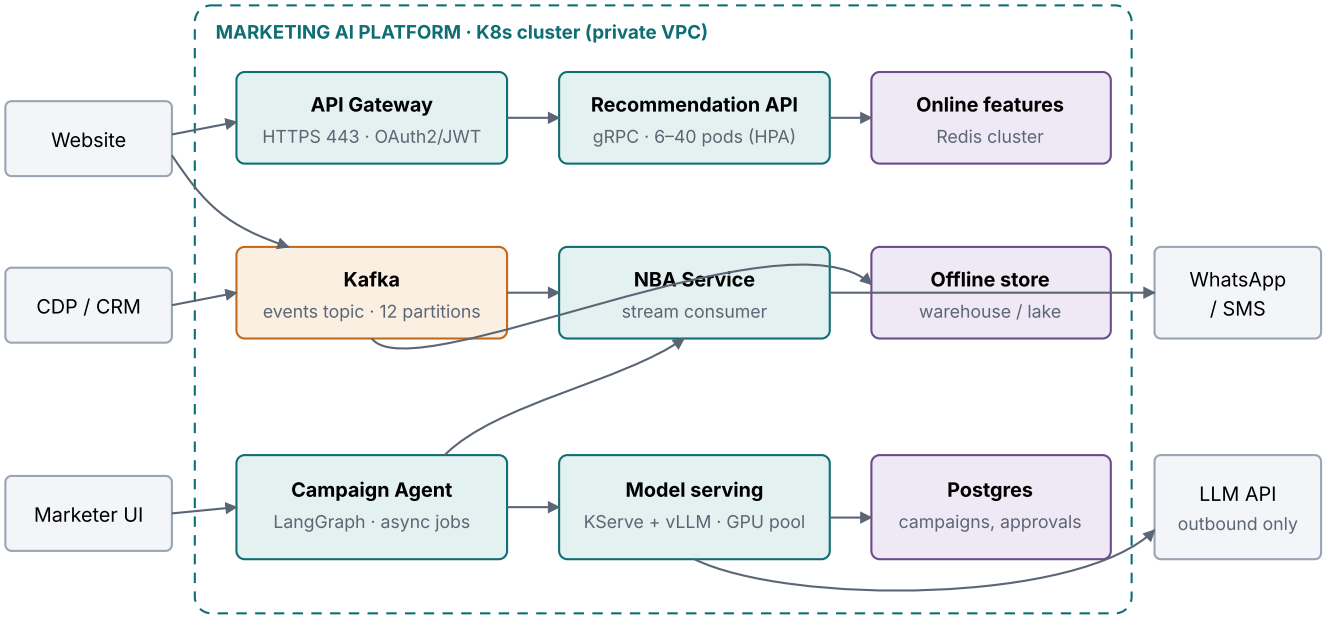
8.2 High-level design Slide 21

C1: Context viewpoint

- **System boundaries:** what's inside the platform vs owned by other teams (see the C1 diagram in Chapter 4).
- **Source and destination systems:** sources = CDP events, CRM profiles, product catalogue. Destinations = website/CMS, WhatsApp/SMS gateway, BI dashboards.
- **Components breakdown:** the major blocks inside the boundary.

C2: Containers and the deployment viewpoint

The slide pairs C2 with the deployment viewpoint. Strictly, C4 has a separate supplementary "deployment diagram", but in practice architects often annotate the container diagram with deployment details, as below.



C2 container view annotated with deployment details (ports, protocols, scaling, node pools). Illustrative.

Checklist from the slide, with example answers:

Item	Example answer
Containers	As in the diagram: gateway, Recommendation API, NBA service, Campaign Agent, model serving, Kafka, Redis, Postgres, warehouse.
Nodes	General node pool (8 × 8 vCPU) + GPU pool (2 × L4 or A10, scale to 4 in peak) for self-hosted models.
Ports & protocols	HTTPS 443 at the edge; gRPC internally for low-latency ranking; Kafka 9093 (TLS); Postgres 5432 inside VPC only.
Authentication / authorization	OAuth2/JWT for the marketer UI (SSO); mTLS service-to-service; RBAC: only "campaign_approver" role can launch.
Networking (inbound/outbound)	Inbound only via gateway. Outbound allow-list: WhatsApp/SMS provider and the LLM API, through a NAT with egress logging.
Capacity planning	See the worked calculation below. Also cover DR: second region or zone, RPO/RTO targets.

WORKED CAPACITY CALCULATION (RECOMMENDATION API)

```
Peak sessions/hour          = 50,000
Ranked page views/session   ≈ 4
Peak requests/sec           = 50,000 × 4 / 3,600 ≈ 56 rps
Burst factor (campaign hit) × 5  → design for ≈ 280 rps
One pod (measured in skeleton) ≈ 40 rps at p95 < 80 ms
Pods needed                 = 280 / 40 = 7  → +30% headroom → 9–10 pods
HPA range                   = min 3 (normal), max 12
```

The "40 rps per pod" figure must come from measuring the skeleton system, not from a guess. This is another reason step 5 of the process matters.

8.3 Information viewpoint / data pipelines Slide 22

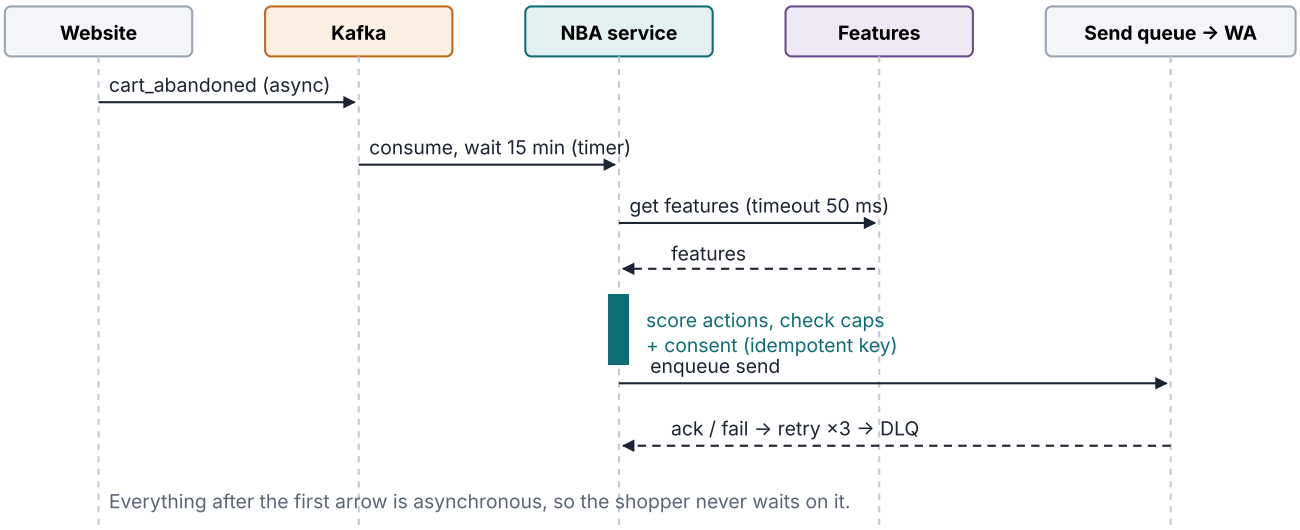
The slide's checklist is answered below for one data flow: **a product view becoming an updated feature**.

```
t=0.0s   Browser fires  {event:"product_view", user:482, sku:"FERT-K-50", ts}
t=0.2s   CDP → Kafka topic "events" (partition = hash(user_id))
t=0.5s   Stream job (Flink) updates  user:482.views_fertilizer_24h  += 1
t=0.6s   Redis online store updated                       ← near real-time
02:00    Nightly Airflow job: events → warehouse → offline features → retrain
```

Slide item	Answer for this project
Data pipelines / flows	Clickstream (stream), orders (CDC from the shop DB), CRM profiles (daily batch), catalogue (on change).
Messaging & queues	Kafka for events; a work queue for campaign sends so a spike doesn't overload the WhatsApp provider.
Velocity	Real-time: online feature lookup. Near real-time: cart-abandon trigger within 15 min. Batch: nightly retrain and audience building.
Size (load / frequency)	E.g. ~3M events/day normal, ~15M on peak days; ~2 KB per event → ~30 GB/day at peak, before compression.
Data quality / preprocessing	Dedup bot traffic, reconcile anonymous → logged-in IDs, validate schemas at ingestion (schema registry).
Retention / backups / replication	Raw events 13 months, aggregated features 24 months, Postgres PITR backups, Kafka replication factor 3.
Sources & ownership	CRM (CRM team), events/CDP (data eng), catalogue (e-commerce team), vector DB of product descriptions (this platform), SQL DBs for campaigns (this platform).
Classification & privacy	Tag fields as PII / sensitive / public. Consent flag checked before any targeting. Phone numbers tokenized and never sent to the LLM. Data residency per PDPL.

8.4 Concurrency viewpoint Slide 23

Here you use sequence diagrams per use case to find concurrency requirements and gaps. Below is the NBA cart-abandonment flow traced end to end:



Sequence diagram: cart abandonment → next-best action → WhatsApp send.

Slide question	Answer surfaced by the diagram
Processes / threads / workers per container	NBA service = Kafka consumer group, 1 worker per partition (12). Recommendation API = stateless pods. Agent = job workers.
Sync vs async	Homepage ranking = sync (user waits). NBA and campaigns = async via events and queues.
Shared state & resources	The weekly message counter is shared. Two workers could both see "1 message" and both send. Fix: atomic increment in Redis, or a DB transaction with an idempotency key per (user, day).
Parallelism & scaling	Partition by user_id so one user's events stay in order. Autoscale consumers on lag.
Rate limits	WhatsApp provider: e.g. 80 msg/s, enforced with a token bucket on the send queue. LLM API: tokens/min quota, so campaign copy generation is batched and cached.
Timeouts, retries, failures	50 ms feature timeout → fall back to the "do nothing" action. Sends: 3 retries with exponential backoff, then **DLQ** (dead-letter queue) for inspection. Circuit breaker on the LLM API.
Scheduling & orchestration	15-min abandonment timer; nightly retrain (Airflow); campaign send windows (no messages 23:00–08:00).

8.5 Operational viewpoint Slide 24

Observability layer

- **Grafana**: resource utilization dashboards (CPU, GPU, memory, pod counts).
- **Loki**: log aggregation.
- **Prometheus**: metrics and alerting (also ingests OpenTelemetry metrics).
- **Kibana**: dashboards, visualization, log analytics.

Example alert: `recommendation_api p95 > 100ms for 5m` → page on-call.

MLOps layer

- **Langfuse**: LLM traces, prompt versions, cost per call, evaluations.
- **MLflow**: model registry and lifecycle, experiment tracking, versioning.
- **Airflow / Flink / Prefect**: batch scheduling, stream processing, transformations.
- **Model serving**: **KServe** (K8s-native, autoscaling, canary rollouts), **BentoML** (package a model as an API, easy containers), **vLLM** (high-throughput LLM inference with PagedAttention and continuous batching).

TRACE: A BAD MODEL RELEASE, END TO END

```
Mon 10:00  ranker v14 registered in MLflow, canary 10% via KServe
Mon 10:40  Grafana: take-rate on canary 3.1% vs control 5.2%
Mon 10:41  Prometheus alert "canary_takerate_drop > 30%"
Mon 10:45  On-call shifts 100% traffic back to v13 (one config change)
Mon 11:30  Langfuse/feature logs show v14 trained on a day of bot traffic
           → add bot filter to data-quality step; ADR updated
```

Each tool on the slide has a specific role in that story. That is how to justify the operational viewpoint to stakeholders.

CHAPTER 9

Architecture governance

Slides 25–26

Governance makes sure that what gets built still matches the design, and that the design still matches the business. The slide gives five key points.

Key point	What it means in practice
Always connect to business goals	Every principle and every major decision should trace to an L1/L2 goal. If it doesn't, ask why it exists.
Principles & standards	Approved technologies, patterns and reference architectures, e.g. "Kafka is the approved event bus", "all LLM calls go through the AI gateway".
Architecture Review Board (ARB)	A small group that reviews designs at key gates (e.g. before build, before go-live) and approves exceptions to standards.
Decision records (ADRs)	A short record per significant decision: context, options, trade-offs, decision, owner. It captures *why*, which code can't.
Light-touch automation	"Guardrails over gates": automate checks in CI/CD instead of making every team wait for a meeting. (Optional.)

Anatomy of an ADR

```
ADR-007  Real-time vs batch scoring for homepage personalization
Status   Accepted · 2026-10-25 · Owner: platform architect
Context  Homepage must render in <300 ms total; ranking budget 80 ms.
         60% of traffic is returning users; catalogue changes ~hourly.
Options  A) Nightly batch: precompute top-50 per user, store in Redis
         B) Real-time: rank at request time from online features
         C) Hybrid: batch candidates nightly + light real-time re-rank
Trade-offs
         A cheapest, but ignores today's session behaviour
         B freshest, but needs online feature store + more pods
         C ~90% of B's freshness at ~40% of B's serving cost
Decision C. Revisit if take-rate uplift from in-session signals > 15%.
Consequences  Need streaming features for session signals (ADR-004).
```

Guardrails over gates: an example

Instead of an ARB meeting for every change, a CI check enforces a principle automatically:

```
# ci/check_no_llm_in_sync_path.py   (illustrative)
SYNC_SERVICES = ["recommendation-api", "api-gateway"]
FORBIDDEN     = ["openai", "anthropic", "llm_client"]

for svc in SYNC_SERVICES:
    deps = read_dependencies(f"services/{svc}/pyproject.toml")
    bad  = [d for d in deps if any(f in d for f in FORBIDDEN)]
    if bad:
        fail(f"{svc} imports {bad}: violates Principle P2 "
             f"'No LLM in the synchronous user path'. Raise an ARB exception.")
```

Input: a PR that adds `openai` to `recommendation-api`. Output: the build fails, with a message naming the principle and the exception path.

CHAPTER 10

Architecture and agentic coding

Slides 27–28

The slide lists three topics: Claude as an architect, the skills needed, and a swarm of agents. The slides only name these topics, so the notes below are one reasonable way to apply them, not a summary of what was said in the session.

An AI assistant as architect's partner

- Draft C1/C2 diagrams as Mermaid from a requirements doc, then refine them by hand.
- Generate the perspectives × views matrix and flag empty or risky cells.
- Write first-draft ADRs: you supply context and options, the model expands trade-offs, and you decide.
- Review a design against your principles (the "agent reviewer" guardrail in the W1 assignment).
- Do the capacity arithmetic in a spreadsheet you can check.

Skills the architect still needs

- Framing the problem and the constraints. The model can't interview your DPO.
- Judging trade-offs against business goals.
- Spotting plausible-but-wrong output: invented tool features, wrong numbers.
- Writing precise context (requirements, principles, ADR templates) so the agent's output is grounded.
- Owning the decision (the "Accountable" trait).

A "swarm" of agents for architecture work

```
Orchestrator  ── splits the review by viewpoint
  ├─ Security agent      checks information × security cells, PDPL, PII in prompts
  ├─ Performance agent   checks latency budgets, capacity math, sync paths
  ├─ Cost agent          estimates LLM tokens/month, GPU hours, egress
  └─ Governance agent    checks every decision has an ADR and links to a goal
Orchestrator  ── merges findings → one ARB-ready review, human signs off
```

This mirrors the views/perspectives structure: each agent is effectively one perspective applied across all views.

CHAPTER 11

Week 1 mini project: a worked starter

Slides 29, 36

Task: a governance pack for the Marketing Automation with AI & Agents system. **Deliver:** one wiki page and one draw.io diagram, max 2 pages. The material below is a starting point to adapt and make your own, not a finished submission.

Part 1: Five architecture principles, each linked to a business goal

#	Principle	Linked goal	Implication
P1	**Consent first:** no personal data is used for targeting without a valid, centrally checked consent record.	PDPL compliance; CSAT 70%	Single consent API; every send path calls it.
P2	**No LLM in the synchronous user path.**	Latency <100 ms; take-rate	LLM output is pre-generated and cached.
P3	**Humans approve what customers see** from generative AI.	Business control; brand safety; CSAT	Approval gate in the campaign agent.
P4	**Every recommendation is attributable** (logged with an ID, model version and outcome).	Measure take-rate +80%; feedback loop	recommendation_id end to end; outcome events.
P5	**Buy commodity, build differentiators.**	Time-to-value; cost	Use managed messaging and LLM APIs; build the NBA logic.

Part 2: Three ADR sketches

ADR-001: API-hosted vs self-hosted LLM for campaign copy

Options: (A) commercial LLM API; (B) self-hosted open model on vLLM; (C) API now, self-host later behind an internal AI gateway.
Trade-offs: A = best Arabic quality and zero ops, but data leaves the boundary and costs scale with volume. B = data stays inside and cost is fixed, but needs a GPU pool, ops skills and possibly lower quality. C = fast start, and the gateway keeps the switch cheap.
Decision: C. Send only anonymized segment descriptions, never PII (P1). Revisit when monthly token cost exceeds the GPU pool cost.

ADR-002: Vector DB choice (product and content embeddings)

Options: pgvector in the existing Postgres; a dedicated vector DB (e.g. Qdrant, Milvus); a managed vector service.
Trade-offs: pgvector = no new infrastructure and transactional joins with stock/price, but limits at very large scale. Dedicated = better filtering and scale, but one more system to run. Managed = least ops, but residency and cost concerns.
Decision: pgvector for ~2M items (P5), with a spike-defined trigger to migrate (e.g. p95 filtered search > 50 ms).

ADR-003: Real-time vs batch scoring

See the full example in Chapter 9 (hybrid: nightly candidates + real-time re-rank).

Part 3: ARB checklist

Area	Check (yes / no / N/A + evidence link)
Security	Threat model done incl. prompt injection on agent tools? Least-privilege for agent tool access? Secrets in a vault? mTLS internally?
Privacy (PDPL)	Data classified (PII/sensitive)? Consent checked before targeting? No PII in prompts or LLM logs? Retention periods set? Residency confirmed? Opt-out honoured within SLA?
Evals	Offline metrics for ranker? A/B test design for uplift? LLM copy evaluated (LLM-as-judge + human sample) for tone, accuracy, Arabic quality? Regression set before each release?
Cost	Token cost per campaign estimated? GPU utilization target? Budget alerts? Cost per incremental sale tracked?
Human oversight	Approval gate before any generated content reaches customers? Kill switch per campaign? Clear owner for each agent? Audit log of approvals?
Operability	Dashboards and alerts defined? Rollback path for models and prompts? Runbook exists?

Part 4 (optional): one automated guardrail

Either the CI check from Chapter 9, or an **agent reviewer**: a prompt that receives your design doc plus the five principles and must return, for each principle, `PASS / FAIL / UNCLEAR` with the quoted evidence. Fail the pipeline on any FAIL.

```
INPUT   design.md + principles.md
OUTPUT  P1 PASS  "every send path calls consent-api (§4.2)"
        P2 FAIL  "§3.1 homepage calls copy-generator synchronously"
        P3 PASS  "approval gate in campaign flow (§5)"
        P4 UNCLEAR "recommendation_id not mentioned for email channel"
        P5 PASS
RESULT  ✗ 1 failure → block merge, notify architect
```

Part 5: The draw.io diagram

Fit it on one page: the C1 context diagram (Chapter 4) or the annotated C2 (Chapter 8). Mark the approval gate, the consent check, and the trust boundary where data leaves for the LLM API. Those three marks visually connect the diagram to P1–P3.

CHAPTER 12

Glossary & self-check

Term	Definition
ADR	Architecture Decision Record: a short document capturing one decision's context, options, trade-offs, decision and owner.
ADM	TOGAF's Architecture Development Method, an iterative cycle of phases from vision to change management.
ARB	Architecture Review Board: reviews designs at gates and approves exceptions.
C4	Context, Containers, Components, Code: four zoom levels of software diagrams.
Cold start	Making useful predictions for a user or item with no history.
DLQ	Dead-letter queue: where messages go after repeated processing failures, for inspection.
Feature store	A system serving consistent precomputed features online (low latency) and offline (training).
NBA	Next-best action: choosing the most valuable action for a user at a moment.
Perspective	A cross-cutting quality concern (security, performance…) applied across all views.
PDPL	Personal Data Protection Law (e.g. Egypt Law 151/2020; Saudi Arabia's PDPL).
Skeleton system	A thin, end-to-end, production-shaped implementation proving the architecture works.
Spike	A time-boxed research task to reduce an unknown.
View / viewpoint	A view is one slice of the architecture for one set of concerns. A viewpoint is the template for producing that kind of view.
Zachman	A 6×6 classification grid of enterprise artifacts (what/how/where/who/when/why × planner…user).

Self-check questions

1. A stakeholder says "the system must be fast". Rewrite it as a testable non-functional requirement for the homepage and for campaigns.
2. What is the difference between a *view* and a *perspective*? Give one example of each from the Marketing Automation system.
3. Why would you choose C4 over Zachman when presenting to a development team, and Zachman over C4 when auditing enterprise coverage?
4. A spike on CDP streaming latency won't finish for two weeks. What do you write in the architecture document in the meantime?
5. Two NBA workers process events for the same user at the same moment and both send a WhatsApp message, exceeding the weekly cap. Which viewpoint should have caught this, and what is the fix?

6. Compute the pod count if peak rises to 80k sessions/hour and one pod handles 50 rps (use the method in §8.2).
7. Write an ADR for "Kafka vs a managed queue service" in under 12 lines.
8. Which of your five principles would an automated CI check enforce most easily, and which needs a human ARB review? Why?

> **ANSWER HINTS**
>
> (1) "p95 ranking <80 ms at 280 rps; campaign of 100k sends completes within 2 h". (5) Concurrency viewpoint, shared state: atomic counter or idempotency key. (6) 80,000×4/3,600 ≈ 89 rps × 5 burst ≈ 445 rps ÷ 50 ≈ 9 pods, +30% headroom ≈ 12.